

نوع داده تاپل در پایتون

در زبان برنامه‌نویسی پایتون، تاپل شبیه به لیست است. تفاوت بین لیست و تاپل در آن است که عناصر تاپل پس از آنکه به آن تخصیص داده شدند، قابل تغییر نخواهند بود. این در حالی است که عناصر لیست قابل تغییر هستند.

مزایای تاپل نسبت به لیست در پایتون

- از نوع داده تاپل در پایتون معمولاً برای نوع داده‌های ناهمگون (متفاوت) استفاده می‌شود و لیست برای نوع داده‌های همگون (مشابه) استفاده می‌شود.
- از آنجا که تاپل غیر قابل تغییر است، حلقه زدن در تاپل (تکرار)، سریع‌تر از لیست انجام می‌شود. بنابراین، کارایی به میزان کمی تقویت می‌شود.
- به دلیل آنکه تاپل‌ها حاوی عناصر غیر قابل تغییر هستند، می‌توان از آن‌ها به عنوان کلید دیکشنری استفاده کرد. انجام چنین کاری با استفاده از لیست امکان‌پذیر نیست.
- تاپل‌ها حاوی عناصر غیر قابل تغییری هستند که می‌توان از آن‌ها به عنوان کلید دیکشنری استفاده کرد. با استفاده از لیست، این کار امکان‌پذیر نیست.
- اگر کاربر داده‌هایی داشته باشند که تغییر نمی‌کنند، پیاده‌سازی آن به عنوان تاپل، تضمین می‌کند که داده‌ها در مقابل نوشتن امن هستند (write-protected) و فقط خواندنی باقی می‌مانند.

ساخت یک تاپل در پایتون

یک تاپل، با قرار دادن همه عناصر درون یک جفت پرانتز، که با کاما از هم جدا شده‌اند، ساخته می‌شود. استفاده از پرانتز اختیاری محسوب می‌شود، ولی استفاده از آن‌ها بهتر است.

```
1 # empty tuple
2 # Output: ()
3 my_tuple = ()
4 print(my_tuple)
5
6 # tuple having integers
7 # Output: (1, 2, 3)
8 my_tuple = (1, 2, 3)
9 print(my_tuple)
10
11 # tuple with mixed datatypes
12 # Output: (1, "Hello", 3.4)
13 my_tuple = (1, "Hello", 3.4)
14 print(my_tuple)
15
16 # nested tuple
17 # Output: ("mouse", [8, 4, 6], (1, 2, 3))
18 my_tuple = ("mouse", [8, 4, 6], (1, 2, 3))
19 print(my_tuple)
20
21 # tuple can be created without parentheses
22 # also called tuple packing
23 # Output: 3, 4.6, "dog"
24
25 my_tuple = 3, 4.6, "dog"
26 print(my_tuple)
27
28 # tuple unpacking is also possible
29 # Output:
30 # 3
31 # 4.6
32 # dog
33 a, b, c = my_tuple
34 print(a)
35 print(b)
36 print(c)
```

دسترسی به یک عنصر تاپل در پایتون

اندیس گذاری

می توان از عملگر `اندیس []`، برای دسترسی به یک عنصر در تاپل استفاده کرد. نکته مهمی که باید به آن توجه داشت آن است که، اندیس عناصر نوع داده تاپل در پایتون از صفر آغاز می شود. بنابراین، اندیس عناصر تاپلی که حاوی ۶ عنصر است از ۰ تا ۵ خواهد بود. تلاش برای دسترسی به عناصر با اندیس های دیگر (۶، ۷ و ...) در چنین تاپلی، منجر به [خطای `IndexError`](#) می شود. اندیس باید یک مقدار صحیح باشد؛ به عبارت دیگر، نمی توان از نوع داده اعشاری یا دیگر انواع داده استفاده کرد.

```
1 my_tuple = ('p','e','r','m','i','t')
2
3 # Output: 'p'
4 print(my_tuple[0])
5
6 # Output: 't'
7 print(my_tuple[5])
8
9 # index must be in range
10 # If you uncomment line 14,
11 # you will get an error.
12 # IndexError: list index out of range
13
14 #print(my_tuple[6])
15
16 # index must be an integer
17 # If you uncomment line 21,
18 # you will get an error.
19 # TypeError: list indices must be integers, not float
20
21 #my_tuple[2.0]
22
23 # nested tuple
24 n_tuple = ("mouse", [8, 4, 6], (1, 2, 3))
25
26 # nested index
27 # Output: 's'
28 print(n_tuple[0][3])
29
30 # nested index
31 # Output: 4
32 print(n_tuple[1][1])
```

اندیس گذاری منفی

پایتون امکان اندیس گذاری منفی را برای دنباله‌ها فراهم می‌کند. اندیس ۱- به آخرین عنصر آرایه و ۲- به عنصر یکی مانده به آخر اشاره دارد. برای سایر عناصر، همین روال ادامه خواهد داشت. مثال‌های زیر در همین رابطه قابل توجه هستند.

```
1 my_tuple = ('p','e','r','m','i','t')
2
3 # Output: 't'
4 print(my_tuple[-1])
5
6 # Output: 'p'
7 print(my_tuple[-6])
```

برش‌زنی

می‌توان به رنجی از عناصر یک تاپل، به وسیله عملگر برش‌زنی (Slicing)، یعنی «دو نقطه (Colon)» دسترسی داشت. در ادامه، مثال‌هایی در همین رابطه، ارائه شده است.

```
1 my_tuple = ('p','r','o','g','r','a','m','i','z')
2
3 # elements 2nd to 4th
4 # Output: ('r', 'o', 'g')
5 print(my_tuple[1:4])
6
7 # elements beginning to 2nd
8 # Output: ('p', 'r')
9 print(my_tuple[: -7])
10
11 # elements 8th to end
12 # Output: ('i', 'z')
13 print(my_tuple[7:])
14
15 # elements beginning to end
16 # Output: ('p', 'r', 'o', 'g', 'r', 'a', 'm', 'i', 'z')
17 print(my_tuple[:])
```

تغییر یک تاپل

برخلاف لیست‌ها، نوع داده تاپل در پایتون غیر قابل تغییر است. این یعنی، عناصر یک تاپل پس از آنکه به تاپل تخصیص پیدا کردند، قابل تغییر محسوب نمی‌شوند. اما اگر عنصر خود نوع داده قابل تغییری مانند لیست باشد، این عصر تو در تو قابل تغییر خواهد بود.

```
1 my_tuple = (4, 2, 3, [6, 5])
2
3 # we cannot change an element
4 # If you uncomment line 8
5 # you will get an error:
6 # TypeError: 'tuple' object does not support item assignment
7
8 #my_tuple[1] = 9
9
10 # but item of mutable element can be changed
11 # Output: (4, 2, 3, [9, 5])
12 my_tuple[3][0] = 9
13 print(my_tuple)
14
15 # tuples can be reassigned
16 # Output: ('p', 'r', 'o', 'g', 'r', 'a', 'm', 'i', 'z')
17 my_tuple = ('p','r','o','g','r','a','m','i','z')
18 print(my_tuple)
```

در کد بالا، ابتدا یک تاپل با **my_tuple** تعریف شده که حاوی سه نوع داده عدد صحیح و یک لیست است. در صورتی که هشتمین خط کد بالا (**my_tuple[1] = 9**) از حالت توضیحات خارج شود، خطای زیر نمایش داده خواهد شد.

```
1 TypeError: 'tuple' object does not support item assignment
```

با توجه به اینکه چهارمین عنصر تاپل، خود یک لیست است که از انواع داده قابل تغییر محسوب می‌شود، با اجرای دستور **my_tuple[3]** به اندیس ۰ از لیست تخصیص پیدا می‌کند. خروجی **print(my_tuple)** در این حالت، به صورت زیر خواهد بود.

```
1 (4, 2, 3, [9, 5])
```

حذف یک تاپل

چنانچه در بالا بیان شد، نمی‌توان عناصر موجود در تاپل را تغییر داد؛ همچنین، نمی‌توان یک عنصر را از تاپل حذف کرد. اما، حذف کردن یک تاپل به طور کامل، با استفاده از کلیدواژه `del` امکان‌پذیر است.

```
1 my_tuple = ('p','r','o','g','r','a','m','i','z')
2
3 # can't delete items
4 # if you uncomment line 8,
5 # you will get an error:
6 # TypeError: 'tuple' object doesn't support item deletion
7
8 #del my_tuple[3]
9
10 # can delete entire tuple
11 # NameError: name 'my_tuple' is not defined
12 del my_tuple
13 my_tuple
```

متدهای نوع داده تاپل در پایتون

متدهایی که عنصر اضافه و یا حذف می‌کنند، برای تاپل وجود ندارند. در واقع، تنها دو متد زیر برای تاپل موجود است.

- `count(x)` : تعداد عناصر تاپل که برابر با آرگومان `x` هستند را باز می‌گرداند.
- `index(x)` [تابع Index در پایتون](#)، اندیس اولین عنصری که با آرگومان `x` مطابقت داشت را باز می‌گرداند.

```
1 my_tuple = ('a','p','p','l','e',)
2
3 # Count
4 # Output: 2
5 print(my_tuple.count('p'))
6
7 # Index
8 # Output: 3
9 print(my_tuple.index('l'))
```

توابع توکار برای تاپل‌ها

- `enumerate()` یک شی شمارشی را باز می‌گرداند. حاوی اندیس و مقدار همه عناصر، به صورت جفت‌های (اندیس، مقدار) است.
- `len()` طول تاپل (تعداد عناصر آن) را باز می‌گرداند.
- `max()` بزرگترین عنصر در تاپل را باز می‌گرداند.
- `min()` کوچکترین عنصر در تاپل را باز می‌گرداند.
- `sorted()` عناصر تاپل را دریافت می‌کند و یک لیست مرتب شده جدید را باز می‌گرداند (خود لیست اصلی را مرتب نمی‌کند).
- `sum()` مجموع همه عناصر تاپل را باز می‌گرداند.
- `tuple()` یک مقدار قابل تکرار (لیست، رشته، دیکشنری و [مجموعه‌های پایتون](#)) را به تاپل تبدیل می‌کند.

SET در پایتون چیست؟

SET در پایتون، مجموعه‌ای نامرتب از عناصر یکتا است. ویژگی‌های SET را می‌توان در سه مورد زیر، خلاصه کرد.

1. در SET، عناصر تکراری نمی‌توانند وجود داشته باشند. اگر عنصری چندین بار به SET اضافه شود، در نهایت فقط یک مورد از آن در SET ذخیره خواهد شد.
2. عناصر درون SET یا مجموعه، نامرتب هستند. ترتیب عناصر درون مجموعه با هربار استفاده از آن تغییر می‌کند.
3. بعد از آن که مجموعه ساخته شد مقادیر درون آن دیگر قابل تغییر نیستند. عناصر درون مجموعه را نمی‌توان تغییر داد حتی اگر متغیرهای حاوی مقدار باشند. اما خود این عناصر را می‌توان حذف یا اضافه کرد. خود مجموعه قابل تغییر است.

SET در پایتون چگونه تعریف می‌شود؟

برای تعریف SET یا مجموعه در پایتون کافی است که عناصر مورد نظر را درون آکولادهای باز و بسته «{}» قرار دهیم و با کمک ویرگول از همدیگر جدا کنیم. همچنین می‌توانیم از متد تعریف شده `set()` در پایتون برای انجام این کار استفاده کنیم. این متد برای تولید مجموعه از طریق توالی ارسال شده به آن به عنوان آرگومان به کار برده می‌شود.

```
1 Days = {"Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"}
2 print(Days)
3 print(type(Days))
4 print("looping through the set elements ... ")
5 for i in Days:
6     print(i)
```

خروجی حاصل از اجرای کد بالا به صورت زیر است.

```
{'Friday', 'Tuesday', 'Monday', 'Saturday', 'Thursday', 'Sunday', 'Wednesday'}
<class 'set'>
looping through the set elements ...
Friday
Tuesday
Monday
Saturday
Thursday
Sunday
Wednesday
```


از آنجا که مجموعه‌ها می‌توانند هر نوع عنصری - به غیر موارد خاص که در ادامه گفته می‌شود - را مانند [اعداد صحیح](#)، [اعداد](#) [Float](#)، تاپل‌ها و غیره را در خود نگهدارند، متد `set()` هم می‌تواند هر نوعی را به عنوان ورودی دریافت کند. اما نکته اینجاست که مجموعه‌ها قادر به نگهداری انواع داده «قابل تغییر (Mutable)» مانند لیست‌ها و دیکشنری‌ها نیستند.

```
1 # Creating a set which have immutable elements
2 set1 = {1,2,3, "JavaTpoint", 20.5, 14}
3 print(type(set1))
4 #Creating a set which have mutable element
5 set2 = {1,2,3,["Javatpoint",4]}
6 print(type(set2))
```

بعد از اجرای کد بالا، خروجی زیر برای نمایش به کاربر بر روی کنسول پایتون چاپ می‌شود.

```
<class 'set'>

Traceback (most recent call last)
<ipython-input-5-9605bb6fbc68> in <module>
4
5 #Creating a set which holds mutable elements
----> 6 set2 = {1,2,3,["Javatpoint",4]}
7 print(type(set2))

TypeError: unhashable type: 'list'
```

نکته: ساخت مجموعه تهی کمی تفاوت دارد. زیرا آکولادها برای تعریف [دیکشنری در پایتون](#) نیز به کار برده می‌شوند. بنابراین پایتون متد `set()` را ارائه داده است. با کمک این متد می‌توان مجموعه تهی ایجاد کرد. فقط کافی است، در زمان استفاده از متد `set()` هیچ آرگومانی به آن ارسال نکنیم.

```
1 # Empty curly braces will create dictionary
2 set3 = {}
3 print(type(set3))
4
5 # Empty set using set() function
6 set4 = set()
7 print(type(set4))
```

خروجی حاصل از اجرای کد بالا به صورت زیر به کاربر نمایش داده می‌شود.

```
<class 'dict'>
<class 'set'>
```

در کد پایین برای ساخت SET در پایتون از آکولاد استفاده کرده‌ایم. به این آکولاد عناصری ارسال شده‌اند که بعضی از آن‌ها تکراری هستند.

```
1 set5 = {1,2,4,4,5,8,9,9,10}
2 print("Return set with unique elements:",set5)
```

بعد از اجرای کد بالا، مجموعه ساخته شده در نهایت به شکل زیر به کاربر نمایش داده می‌شود. البته مجموعه نمایش داده شده همان شکل واقعی مجموعه ساخته شده نهایی است.

```
Return set with unique elements: {1, 2, 4, 5, 8, 9, 10}
```

در کد بالا می‌توان دید که مجموعه `set5` از چندین عنصر تکراری تشکیل شده است. اما بعد از اینکه مجموعه ساخته شده را چاپ می‌کنیم، تمام اعضای تکراری حذف شده‌اند.

اضافه کردن عنصر به SET در پایتون

پایتون متدهای Add و Update را برای اضافه کردن عناصر مشخص شده به مجموعه‌ها ارائه داده است. متد add() برای افزودن عنصرها به صورت مجزا به کاربرده می‌شود در حالی که از متد update() برای افزودن چندین عنصر به صورت همزمان به مجموعه استفاده می‌شود. در مثال‌های این بخش استفاده از هر دو متد را نمایش داده‌ایم.

```
1 Months = set(["January","February", "March", "April", "May", "June"])
2 print("\nprinting the original set ... ")
3 print(months)
4 print("\nAdding other months to the set...");
5 Months.add("July");
6 Months.add ("August");
7 print("\nPrinting the modified set...");
8 print(Months)
9 print("\nlooping through the set elements ... ")
10 for i in Months:
11     print(i)
```

بعد از اجرای کدهای بالا، خروجی زیر به عنوان نتیجه به کاربر نمایش داده می‌شود.

```
printing the original set ...
{'February', 'May', 'April', 'March', 'June', 'January'}

Adding other months to the set...

Printing the modified set...
{'February', 'July', 'May', 'April', 'March', 'August', 'June', 'January'}

looping through the set elements ...
February
July
May
April
March
August
June
January
```

مثال دوم: استفاده از متد Update

پایتون از متد `update()` برای اضافه کردن تعداد عناصر بیشتر از یک مورد به مجموعه‌ها استفاده می‌کند. متد `update()` به عنوان آرگومان می‌تواند اشیاء پیمایش‌پذیر را هم دریافت کند. در کد زیر، مثالی را مشاهده می‌کنید که در آن به عنوان آرگومان ورودی این متد از لیست استفاده شده است.

```
1 Months = set(["January","February", "March", "April", "May", "June"])
2 print("\nprinting the original set ... ")
3 print(Months)
4 print("\nupdating the original set ... ")
5 Months.update(["July","August","September","October"])
6 print("\nprinting the modified set ... ")
7 print(Months);
```

بعد از اجرای کدهای بالا، خروجی زیر به عنوان نتیجه به کاربر نمایش داده می‌شود.

```
printing the original set ...
{'January', 'February', 'April', 'May', 'June', 'March'}

updating the original set ...
printing the modified set ...
{'January', 'February', 'April', 'August', 'October', 'May', 'June', 'July',
'September', 'March'}
```

حذف عناصر از SET در پایتون

پایتون متدهایی را معرفی کرده است که برای حذف عنصر از مجموعه قابل استفاده‌اند. این ابزار، متدهای `Remove` و `Discard` هستند. اما این دو متد نیز با یکدیگر تفاوت دارند. در صورت عدم وجود عنصر مورد نظر در مجموعه، استفاده از متد `Discard` باعث می‌شود که مجموعه به همان حالت قبلی و بدون تغییر باقی‌بماند. اما اگر از متد `Remove` استفاده کنیم، درحالی که عنصر مورد نظر در مجموعه وجود ندارد، باعث ایجاد خطا در برنامه خواهد شد.

مثال اول: استفاده از متد Discard

در این مثال هم از مجموعه ماه‌های سال months استفاده می‌کنیم. با کمک متد `discard()` اقدام به حذف عناصر موجود در SET کرده‌ایم.

```
1 months = set(["January", "February", "March", "April", "May", "June"])
2 print("\nprinting the original set ... ")
3 print(months)
4 print("\nRemoving some months from the set...");
5 months.discard("January");
6 months.discard("May");
7 print("\nPrinting the modified set...");
8 print(months)
9 print("\nlooping through the set elements ... ")
10 for i in months:
11     print(i)
```

بعد از اجرای کدهای بالا، خروجی زیر به عنوان نتیجه به کاربر نمایش داده می‌شود.

```
printing the original set ...
{'February', 'January', 'March', 'April', 'June', 'May'}

Removing some months from the set...

Printing the modified set...
{'February', 'March', 'April', 'June'}

looping through the set elements ...
February
March
April
June
```

مثال دوم: استفاده از متد Remove

پایتون متد `remove()` را هم برای حذف عناصر معرفی کرده است. از این متد بر روی عناصر مجموعه نیز می‌توان استفاده کرد. در مثال زیر، با استفاده از متد `remove()` عناصری را از مجموعه `months` حذف کرده‌ایم.

```
1 months = set(["January", "February", "March", "April", "May", "June"])
2 print("\nprinting the original set ... ")
3 print(months)
4 print("\nRemoving some months from the set...");
5 months.remove("January");
6 months.remove("May");
7 print("\nPrinting the modified set...");
8 print(months)
```

بعد از اجرای کدهای بالا، خروجی زیر به عنوان نتیجه به کاربر نمایش داده می‌شود.

```
printing the original set ...
{'February', 'June', 'April', 'May', 'January', 'March'}

Removing some months from the set...

Printing the modified set...
{'February', 'June', 'April', 'March'}
```

مثال سوم: استفاده از متد Pop

برای حذف عناصر درون SET می‌توان از [متد Pop](#) نیز استفاده کرد. این متد را معمولاً به عنوان عضوی از [متدهای لیست در پایتون](#) می‌شناسند. ولی تقریباً بر روی همه اشیاء پیمایش‌پذیر پایتون قابل استفاده است. به طور کلی، متد `pop()` همیشه آخرین عنصر را حذف می‌کند. اما مجموعه‌ها ساختارهای نامرتبی هستند، بنابراین نمی‌توانیم عنصر حذف شده از مجموعه را از قبل تشخیص داده یا پیش‌بینی کنیم.

```
1 Months = set(["January","February", "March", "April", "May", "June"])
2 print("\nprinting the original set ... ")
3 print(Months)
4 print("\nRemoving some months from the set...");
5 Months.pop();
6 Months.pop();
7 print("\nPrinting the modified set...");
8 print(Months)
```

بعد از اجرای کدهای بالا، خروجی زیر به عنوان نتیجه به کاربر نمایش داده می‌شود.

```
printing the original set ...
{'June', 'January', 'May', 'April', 'February', 'March'}

Removing some months from the set...

Printing the modified set...
{'May', 'April', 'February', 'March'}
```

در کدهای بالا، آخرین عنصر مجموعه Months، ماه March است. اما متد `pop()` ماه‌های June و January را از SET حذف کرده است. این مسئله بخاطر این است که مجموعه‌ها نامرتب هستند. بنابراین متد `pop()` نمی‌تواند آخرین عنصر مجموعه را تشخیص دهد. می‌توان گفت که متد `pop()` به صورت تصادفی گزینه‌هایی را از درون مجموعه‌ها انتخاب کرده و حذف می‌کند. هر بار که کدهای بالا را اجرا کنید با ماه‌های باقیمانده متفاوتی درون مجموعه Months روبه‌رو می‌شوید.

مثال چهارم: متد Clear

پایتون، متد دیگری را نیز با هدف حذف همه عناصر از درون مجموعه ارائه داده است. با کمک متد `clear()` می‌توان همه عناصر را به صورت یکجا از درون مجموعه حذف کرد. در مثال زیر، روش استفاده از این متد را برای پاکسازی مجموعه `Months` نمایش داده‌ایم.

```
1 Months = set(["January", "February", "March", "April", "May", "June"])
2 print("\nprinting the original set ... ")
3 print(Months)
4 print("\nRemoving all the items from the set...");
5 Months.clear()
6 print("\nPrinting the modified set...")
7 print(Months)
```

بعد از اجرای کدهای بالا، خروجی زیر به عنوان نتیجه به کاربر نمایش داده می‌شود.

```
printing the original set ...
{'January', 'May', 'June', 'April', 'March', 'February'}

Removing all the items from the set...

Printing the modified set...
set()
```


تفاوت بین متدهای Remove و Discard

علی‌رغم اینکه متدهای Remove و Discard هر دو وظیفه یکسانی را انجام می‌دهند، اما تفاوت مهمی هم بین این دو متد وجود دارد. اگر عنصری که باید از دورن مجموعه و با استفاده متد Discard حذف شود، در مجموعه وجود نداشته باشد، پایتون هیچ خطایی را به کاربر برنمی‌گرداند. یعنی مجموعه به صورت دست‌نخورده، به همان شکل اول باقی می‌ماند و برنامه هم ادامه مسیر عادی خود را طی کرده و اجرا می‌شود.

در مثال زیر، استفاده از هر دو متد `discard()` و `remove()` را برای حذف عنصر ناموجود از مجموعه نمایش داده‌ایم.

```
1 Months = set(["January", "February", "March", "April", "May", "June"])
2 print("\nprinting the original set ... ")
3 print(Months)
4 print("\nRemoving items through discard() method...");
5 Months.discard("Feb"); #will not give an error although the key feb is not available in
6 print("\nprinting the modified set...")
7 print(Months)
8 print("\nRemoving items through remove() method...");
9 Months.remove("Jan") #will give an error as the key jan is not available in the set.
10 print("\nPrinting the modified set...")
11 print(Months)
```

بعد از اجرای کدهای بالا، خروجی زیر به عنوان نتیجه به کاربر نمایش داده می‌شود.

```
printing the original set ...
{'March', 'January', 'April', 'June', 'February', 'May'}

Removing items through discard() method...

printing the modified set...
{'March', 'January', 'April', 'June', 'February', 'May'}

Removing items through remove() method...
Traceback (most recent call last):
File "set.py", line 9, in
Months.remove("Jan")
KeyError: 'Jan'
```

عملیات رایج بر روی ساختار SET در پایتون

بر روی مجموعه‌های پایتون می‌توان همان عملیات ریاضی رایج . مخصوص مجموعه‌ها را نیز اجرا کرد. این عملیات را در فهرست زیر نام برده‌ایم.

- اجتماع (Union)
- اشتراک (Intersection)
- اختلاف یا تفاضل (Difference)
- تفاضل متقارن (Symmetric Difference)

اجتماع دو مجموعه

برای اینکه در پایتون، تعداد دو با بیشتری مجموعه را با یکدیگر ترکیب کرده و مجموعه جدیدی ایجاد کنیم، از تابع Union استفاده می‌کنیم. تمام ویژگی‌های متمایز و منحصر به فرد هر کدام از مجموعه‌های مورد عملیات، در مجموعه نهایی حضور خواهد داشت. به عنوان پارامتر می‌توان از یک یا تعداد بیشتری مجموعه برای ارسال به تابع Union استفاده کرد.

اگر به تابع Union فقط یک مجموعه ارسال کنیم، به عنوان خروجی نسخه رونوشتی از مجموعه ارائه شده به خود را برمی‌گرداند. اما اگر بیش از یک مجموعه را به این تابع به عنوان پارامتر ارسال کنیم، تابع Union مجموعه جدیدی را برمی‌گرداند. این مجموعه جدید، شامل همه عناصر مختلف از همه آگومان‌ها - مجموعه‌های دریافت شده - می‌شود.

مثال اول: استفاده از عملگر اجتماع

عملگر اجتماع بین مجموعه‌ها با نماد «|» شناخته می‌شود. این عملگر دو عملوند در سمت چپ و راست خود می‌پذیرد. در مثال زیر، استفاده از عملگر | را برای اجرای عملیات اجتماع بین دو مجموعه نمایش داده‌ایم.

```
1 Days1 = {"Monday", "Tuesday", "Wednesday", "Thursday", "Sunday"}
2 Days2 = {"Friday", "Saturday", "Sunday"}
3 print(Days1|Days2) #printing the union of the sets
```

بر اثر اجرای کد بالا، خروجی زیر در کنسول پایتون به کاربر نمایش داده می‌شود.

```
{'Friday', 'Sunday', 'Saturday', 'Tuesday', 'Wednesday', 'Monday', 'Thursday'}
```

مثال دوم: استفاده از متد Union

ابزار اختصاصی پایتون برای محاسبه نتیجه اجتماع دو مجموعه، متد اختصاصی `union()` است. در کادر زیر، با کمک مثال ساده‌ای استفاده از متد `union()` را برای محاسبه اجتماع بین دو مجموعه نمایش داده‌ایم.

```
1 Days1 = {"Monday", "Tuesday", "Wednesday", "Thursday"}
2 Days2 = {"Friday", "Saturday", "Sunday"}
3 print(Days1.union(Days2)) #printing the union of the sets
```

بر اثر اجرای کد بالا، خروجی زیر در کنسول پایتون به کاربر نمایش داده می‌شود.

```
{'Friday', 'Monday', 'Tuesday', 'Thursday', 'Wednesday', 'Sunday', 'Saturday'}
```

البته با استفاده از متد `union()` می‌توان اجتماع بیش از دو مجموعه را نیز محاسبه کرد. برای این کار باید مجموعه‌های بیشتری را به عنوان آرگومان به این متد ارسال کرد. در کد زیر، روش محاسبه اجتماع سه مجموعه مختلف را با استفاده از متد `union()` نمایش داده‌ایم.

```
1 # Create three sets
2 set1 = {1, 2, 3}
3 set2 = {2, 3, 4}
4 set3 = {3, 4, 5}
5
6 # Find the common elements between the three sets
7 common_elements = set1.union(set2, set3)
8
9 # Print the common elements
10 print(common_elements)
```

بر اثر اجرای کد بالا، خروجی زیر در کنسول پایتون به کاربر نمایش داده می‌شود.

```
{1, 2, 3, 4, 5}
```

اشتراک دو مجموعه

برای کشف اینکه بین دو مجموعه مختلف چه عناصر مشترکی وجود دارند، باید از تابع `Intersection` استفاده کنیم. با کمک این متد فقط عناصری در مجموعه جواب نهایی به بیرون برگشت داده می‌شوند که در همه مجموعه‌های مورد مقایسه وجود داشته باشند. به عنوان پارامترهای تابع `Intersection` می‌توان از یک یا چند مجموعه مختلف استفاده کرد.

اگر فقط از یک مجموعه به عنوان پارامتری برای این تابع استفاده کنیم، جواب نهایی رونوشتی از مجموعه مورد نظر خواهد بود. اما اگر از چندین مجموعه مختلف به عنوان پارامتر استفاده کرده و همه را به تابع `intersection()` ارسال کنیم، جواب نهایی به شکل مجموعه‌ای از عناصری خواهد بود که در تمام مجموعه‌های ارسال شده به عنوان پارامتر حضور دارند. در واقع عناصری که بین همه مجموعه‌ها مشترک هستند.

مثال اول: استفاده از عملگر `And`

در این مثال از عملگر `&` برای محاسبه اشتراک بین دو مجموعه استفاده می‌کنیم. توجه دارید که استفاده از عملگر `&` و `and` دقیقاً شبیه به هم است.

```
1 Days1 = {"Monday", "Tuesday", "Wednesday", "Thursday"}
2 Days2 = {"Monday", "Tuesday", "Sunday", "Friday"}
3 print(Days1&Days2) #prints the intersection of the two sets
```

بر اثر اجرای کد بالا، خروجی به صورت `{'Monday', 'Tuesday'}` در کنسول پایتون به کاربر نمایش داده می‌شود.

مثال دوم: استفاده از متد `Intersection`

```
1 set1 = {"Devansh", "John", "David", "Martin"}
2 set2 = {"Steve", "Milan", "David", "Martin"}
3 print(set1.intersection(set2)) #prints the intersection of the two sets
```

بر اثر اجرای کد بالا، خروجی به صورت `{'Martin', 'David'}` در کنسول پایتون به کاربر نمایش داده می‌شود.

مثال سوم: محاسبه اشتراک بیش از دو مجموعه

با کمک تابع `intersection()` می‌توان اشتراک بیش از دو مجموعه را هم محاسبه کرد. این کار شبیه به اجرای عملیات با تابع `union()` برای محاسبه اجتماع بیش از دو مجموعه است. توجه کنید که در این توابع، ترتیب قرار دادن آرگومان‌ها مهم نیست. فقط حتما باید آرگومان اول در سمت چپ یا پشت متد قرار گرفته و با کمک نقطه به آن وصل شود.

```
1 # Create three sets
2 set1 = {1, 2, 3}
3 set2 = {2, 3, 4}
4 set3 = {3, 4, 5}
5
6 # Find the common elements between the three sets
7 common_elements = set1.intersection(set2, set3)
8
9 # Print the common elements
10 print(common_elements)
```

بر اثر اجرای کد بالا، خروجی به صورت `{3}` در کنسول پایتون به کاربر نمایش داده می‌شود.

متد Intersection_Update

متد `intersection_update()` تمام عناصری را که در همه مجموعه‌ها مشترک نیستند از مجموعه اول حذف می‌کند. این متد با متد `intersection()` متفاوت است. زیرا در این متد، مجموعه اصلی را از طریق حذف عناصر نامشترک با سایر مجموعه‌ها تغییر می‌دهیم. از سوی دیگر متد `intersection()` مجموعه جدیدی را برمی‌گرداند.

```
1 a = {"Devansh", "bob", "castle"}
2 b = {"castle", "dude", "emyway"}
3 c = {"fuson", "gaurav", "castle"}
4
5 a.intersection_update(b, c)
6
7 print(a)
```

بر اثر اجرای کد بالا، خروجی به صورت `{'castle'}` در کنسول پایتون به کاربر نمایش داده می‌شود. مجموعه اصلی در این نوع از توابع، همان مجموعه‌ای است که از سمت چپ به تابع متصل می‌شود. تغییرات بر روی آن اعمال می‌شود. سایر متدها به صورت دست‌نخورده باقی می‌مانند.

اختلاف یا تفاضل بین دو مجموعه

برای محاسبه تفاضل بین دو مجموعه هم می‌توان از عملگر منها «-» استفاده کرد و هم از متد `Difference`. فرض کنیم که دو مجموعه `A` و `B` وجود دارند. هدف بدست آوردن تفاضل مجموعه `A` از `B` است. این مسئله به صورت «`A - B`» هم نمایش داده می‌شود. این تفاضل به معنای آن است که مجموعه جواب، شامل تمام عناصری از مجموعه `A` است که در مجموعه `B` وجود ندارند.

مثال اول: استفاده از عملگر منها

عملگر منها به صورت خط تیره «-» نمایش داده می‌شود و یکی از ساده‌ترین روش‌ها برای محاسبه تفاضل مجموعه سمت راست آن از مجموعه سمت چپ آن است. در کادر زیر، روش استفاده از این عملگر را نمایش داده‌ایم.

```
1 Days1 = {"Monday", "Tuesday", "Wednesday", "Thursday"}
2 Days2 = {"Monday", "Tuesday", "Sunday"}
3 print(Days1-Days2) #{"Wednesday", "Thursday" will be printed}
```

بر اثر اجرای کد بالا، خروجی به صورت `{'Thursday', 'Wednesday'}` در کنسول پایتون به کاربر نمایش داده می‌شود.

مثال دوم: استفاده از متد `Difference`

متد `difference()` که در مثال زیر، استفاده شده همان کارکرد عملگر منها را دارد. در مثال زیر، روش استفاده از این متد را برای محاسبه تفاضل دو مجموعه مشاهده می‌کنید.

```
1 Days1 = {"Monday", "Tuesday", "Wednesday", "Thursday"}
2 Days2 = {"Monday", "Tuesday", "Sunday"}
3 print(Days1.difference(Days2)) # prints the difference of the two sets Days1 and Days2
```

بر اثر اجرای کد بالا، خروجی به صورت `{'Thursday', 'Wednesday'}` در کنسول پایتون به کاربر نمایش داده می‌شود.

تفاضل متقارن دو مجموعه

«تفاضل متقارن» (Symmetric Difference) بین دو مجموعه به عملیات کشف عناصری می‌گویند که فقط در یکی از مجموعه‌ها وجود داشته باشند. یعنی اینکه بین هر دو مجموعه مشترک نباشند و فقط در یکی از دو مجموعه وجود داشته باشند. به بیان دیگر تفاضل متقارن دو مجموعه شامل عناصری است که در `set1` یا `set2` وجود داشته باشند اما عضوی از مجموعه اشتراک `set1` و `set2` نباشند.

برای محاسبه تفاضل متقارن دو مجموعه در پایتون از متد `symmetric_difference()` و عملگر « $\wedge$ » استفاده می‌شود. متد `symmetric_difference()` مجموعه جدیدی را در خروجی برمی‌گرداند که شامل همه عناصری است که فقط در یکی از مجموعه‌های صورت مسئله وجود داشته باشند. عناصری که در هر دو مجموعه اولیه وجود دارند، عضوی از مجموعه جواب نخواهند بود.

مثال اول: استفاده از عملگر نماد عملیات تفاضل متقارن

نماد عملگر مربوط به محاسبه تفاضل متقارن را به شکل « $\wedge$ » نمایش می‌دهند. استفاده از این عملگر بسیار ساده است. در کادر زیر، روش استفاده از آن را بر روی مجموعه‌هایی از اعداد صحیح نمایش داده‌ایم.

```
1 a = {1,2,3,4,5,6}
2 b = {1,2,9,8,10}
3 c = a^b
4 print(c)
```

بر اثر اجرای کد بالا، خروجی زیر بر روی صفحه مانیتور کاربر چاپ می‌شود.

```
{3, 4, 5, 6, 8, 9, 10}
```

مثال دوم: استفاده از متد Symmetric_Difference

در این مثال هم از مجموعه‌های نمایش داده شده در مثال قبل استفاده کرده‌ایم. کاملاً قابل مشاهده است که روش استفاده و نتیجه استفاده از هر دو تکنیک یکسان است.

```
1 a = {1,2,3,4,5,6}
2 b = {1,2,9,8,10}
3 c = a.symmetric_difference(b)
4 print(c)
```

بر اثر اجرای کد بالا، خروجی زیر بر روی صفحه مانیتور کاربر چاپ می‌شود.

```
{3, 4, 5, 6, 8, 9, 10}
```